Widgets et mise en page

Séance 2 · état local et formulaires

Point de situation

Séance 1 : Flutter installé, Dart parcouru, le tutoriel officiel commencé.

Aujourd'hui commence **Pantry**, l'application construite jusqu'à la fin du semestre.

Aucun réseau, aucun état partagé, aucune navigation. Uniquement de la mise en page et de la saisie.

Aujourd'hui

Horaire	Bloc
0:00 – 0:15	Rappels
0:15 – 1:00	Widgets, cycle de vie, modèle de layout
1:00 – 2:00	TP 1 · Devine le rendu
2:00 – 2:15	Pause
2:15 – 2:55	`setState` et formulaires
2:55 – 3:55	TP 2 · fiche produit et formulaire
3:55 – 4:15	Débrief

Widgets

Cycle de vie et modèle de contraintes

Le `State` survit, le widget non

`StatelessWidget`, sa description ne dépend que de ses paramètres. Reconstruit quand ils changent.

`StatefulWidget`, associé à un objet `State` qui survit aux reconstructions. C'est le `State` qui porte les données mutables, pas le widget.

Le widget est jeté et recréé à chaque build. Le `State`, non. Cette séparation explique la totalité du cycle de vie qui suit.

Les quatre moments d'un `State`

Méthode	Quand	Équivalent Compose
`initState`	une fois, à l'insertion	`LaunchedEffect(Unit)`
`didUpdateWidget`	à chaque nouveau widget parent	un effet à clé : `LaunchedEffect(param)`
`build`	à chaque reconstruction	le corps du `@Composable`
`dispose`	au retrait définitif	`DisposableEffect(Unit) { onDispose { … } }`

La coroutine d'un `LaunchedEffect` est annulée quand le composable quitte la composition. `initState` est synchrone : un `Future` lancé depuis lui continue après `dispose`.

`mounted` indique si le `State` est encore dans l'arbre. À tester après tout `await` avant d'appeler `setState`.

Seul l'arbre des widgets est recréé à chaque build

Trois arbres parallèles, de durées de vie différentes.

Arbre	Rôle	Durée de vie
Widget	configuration **immuable**	recréé à chaque build
Element	instance montée, fait le lien	persistant tant que le type ne change pas
RenderObject	mesure, positionne, peint	persistant, coûteux

Un widget n'est **pas** l'objet à l'écran. C'est une description, jetable, comparable à un `@Composable` réexécuté.

Reconstruire est bon marché

Allouer des widgets est une opération triviale.

Recalculer une mise en page et repeindre ne l'est pas, et Flutter ne le refait que là où la description a réellement changé.

Un widget `const` arrête la reconstruction venue du parent

```
const SizedBox(height: 16)      // même instance : le parent ne le reconstruit pas
SizedBox(height: 16)            // nouvelle instance à chaque build
```

Un widget `const` est un **objet unique en mémoire**. Quand le parent se reconstruit, Flutter trouve la même instance à la même place et saute toute la sous-arborescence.

Il se reconstruit encore si un `InheritedWidget` qu'il lit change (`Theme.of`, séance 3), ou si son propre `State` appelle `setState`.

`very_good_analysis` signale chaque `const` manquant. Le suivre.

Les contraintes descendent, les tailles remontent

`RenderPadding.performLayout`, à peine simplifié :

```
void performLayout() {
  final padding = _resolvedPadding;
  final inner = constraints.deflate(padding);          // ① contrainte réduite
  child!.layout(inner, parentUsesSize: true);          // ② l'enfant se dimensionne
  (child!.parentData! as BoxParentData).offset =
      Offset(padding.left, padding.top);               // ③ position
  size = constraints.constrain(Size(
    padding.horizontal + child!.size.width,
    padding.vertical + child!.size.height,
  ));
}
```

① les contraintes descendent · ② les tailles remontent · ③ le parent positionne.

`Padding` ne dessine rien : il réduit une contrainte et décale un enfant.

Constraints go down.

Sizes go up.

Parent sets position.

Les contraintes descendent. Les tailles remontent. Le parent positionne.

Un enfant ne connaît jamais sa position

1. Un parent transmet à son enfant des **contraintes** : largeur et hauteur, minimales et maximales.
2. L'enfant choisit sa **taille** dans ces bornes, et la renvoie.
3. Le parent **place** l'enfant dans son propre repère.

Un parent n'impose jamais directement une taille : il impose des bornes. Compose suit le même protocole : `measure(constraints)`, puis `place(x, y)`.

La quasi-totalité des surprises de mise en page vient d'un oubli d'une de ces trois étapes.

La page **`docs.flutter.dev/ui/layout/constraints`** déroule 29 exemples commentés à partir de cette seule règle. C'est la référence à ouvrir dès qu'une mise en page résiste.

Tight, loose, unbounded : la liberté laissée à l'enfant

```
SizedBox(width: 100, child: …)   // tight   : min = max = 100
Center(child: …)                 // loose   : min = 0, max = disponible
```

Contrainte	`min / max`	L'enfant…
tight	`min == max`	n'a aucun choix, sa demande est ignorée
loose	`min == 0`	choisit entre 0 et le maximum
unbounded	`max == infinity`	doit se dimensionner lui-même, sinon erreur

Ce sont les termes de l'API : `BoxConstraints.tight()`, `.loose()`, et `hasBoundedHeight`. Ils apparaissent tels quels dans les messages d'erreur.

Une contrainte `unbounded` transmise à un widget qui veut « tout l'espace » est une erreur, pas une approximation.

`Row` et `Column` laissent l'axe principal libre

Transmettent une contrainte **unbounded** sur leur axe principal : chaque enfant prend ce qu'il veut, la somme est calculée ensuite.

```
Row(children: [
  Expanded(child: A()),            // reçoit une contrainte tight
  Expanded(flex: 2, child: B()),   // deux fois plus large
  const Icon(Icons.star),          // se dimensionne lui-même
])
```

`Expanded` répartit l'espace **restant** au prorata des `flex`.

`Flexible` répartit de la même façon, mais laisse l'enfant prendre moins.

`Expanded` ne dessine rien

C'est une **annotation** posée sur un enfant, lue par le parent.

Seuls les `Flex` (`Row` et `Column`) savent la lire. Dans un `Stack`, l'annotation équivalente est `Positioned`.

```
Incorrect use of ParentDataWidget.
```

Ce message signale une annotation placée sous un parent qui ne la comprend pas. Il désigne le **parent immédiat**, pas un ancêtre lointain.

`Modifier.weight` n'existe que dans `RowScope` et `ColumnScope` : Compose refuse la faute à la compilation, Flutter la signale à l'exécution.

L'écran rouge ne vient que d'une exception dans un `build`

À l'écran	Nature
rayures jaunes et noires	dépassement de place
une zone non peinte	erreur de layout, contrainte impossible
un rendu d'apparence normale	annotation sous le mauvais parent
écran rouge	exception levée pendant un `build`

Dans les quatre cas, l'application continue de tourner, et la console nomme le widget fautif. Un écran juste ne prouve pas un code juste.

Lire les contraintes avant d'ajouter un widget

```dart
import 'package:flutter/rendering.dart';

void main() {
  debugPaintSizeEnabled = true;   // bordures et espacements visibles
  runApp(const PantryApp());
}
```

Le **Widget Inspector** de l'extension Flutter affiche l'arbre et, pour chaque nœud, les contraintes reçues et la taille renvoyée.

Utiliser l'un des deux dès le premier symptôme, avant d'ajouter un widget au hasard.

Pas de `Modifier` : chaque effet est un widget

```
Modifier.padding(16.dp).background(Color.Red)      // Compose
```

```
Padding(                                            // Flutter
  padding: const EdgeInsets.all(16),
  child: ColoredBox(color: Colors.red, child: …),
)
```

Chaque effet est un widget qui en enveloppe un autre : l'arbre est plus profond, mais il n'existe qu'une notion, sans API parallèle.

Le réflexe demande une à deux séances. La page `docs.flutter.dev/flutter-for/compose-devs` en donne d'autres exemples, chacun en regard de son équivalent Compose.

Les composants Material 3 des deux côtés

Même nom en Compose et en Flutter : `Scaffold`, `Card`, `FloatingActionButton`, `NavigationBar`, `SegmentedButton`, `FilterChip`, `Switch`, `Checkbox`.

Compose	Flutter
`TopAppBar`	`AppBar`
`ListItem`	`ListTile`
`Button`	`FilledButton`
`TabRow`	`TabBar` et `TabBarView`
`ExposedDropdownMenuBox`	`DropdownMenu`
`OutlinedTextField`	`TextField` avec `OutlineInputBorder`

Chercher le composant dans `docs.flutter.dev/ui/widgets/material` avant de l'assembler à partir de `Container`.

TP 1 : Devine le rendu

60 minutes

Huit extraits. **Prédiction écrite avant exécution.**

`docs/seance-02-devine-le-rendu.md`

Pause

15 minutes

`setState`

État local et formulaires

`setState` reconstruit tout le `build()`

```
setState(() => _tabIndex = index);
```

Deux choses, dans cet ordre :

1. exécuter la fonction passée, c'est là que l'état change ;
2. marquer le `State` comme à reconstruire.

La portée est le `build()` entier du `State`, pas la partie qui a changé. Un `setState` dans un widget haut placé reconstruit tout ce que son `build()` recrée ; seules les instances inchangées, `const` en tête, sont sautées.

Compose ne recompose que les fonctions `@Composable` qui lisent l'état modifié. D'où, en Flutter, des `State` proches de ce qu'ils modifient.

`setState` ne traverse pas les écrans

- Il ne reconstruit **pas** immédiatement : la reconstruction a lieu à la frame suivante.
- Il ne traverse **pas** les frontières : un autre `State` n'en sait rien.
- Il ne fonctionne **pas** après `dispose`. D'où le test de `mounted` après un `await`.

Le partage d'état entre écrans est le sujet de la séance 5.

Un formulaire en trois pièces

```dart
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: TextFormField(
    validator: (v) => (v?.isEmpty ?? true) ? 'Champ obligatoire' : null,
  ),
)

if (_formKey.currentState!.validate()) { … }
```

Pièce	Rôle
`Form`	regroupe les champs
`GlobalKey<FormState>`	donne accès à `validate()` et `reset()`
`validator`	renvoie `null` si valide, un message sinon

Compose n'a pas d'équivalent de `Form` : la validation s'y écrit à la main, champ par champ.

`validate()` ignore ce qui n'est pas un `FormField`

`validate()` interroge **les `FormField` descendants**, et rien d'autre.

Un sélecteur de date posé à côté du formulaire n'en est pas un : sa valeur ne sera jamais vérifiée, et aucune erreur ne le signalera. `showDatePicker` renvoie un `Future<DateTime?>`, `null` si l'utilisateur annule.

Deux issues :

- contrôler la valeur à la main avant de soumettre ;
- envelopper le sélecteur dans un `FormField<DateTime>`, qui le fait entrer dans le cycle.

Toujours libérer le contrôleur dans `dispose`

```dart
final _controller = TextEditingController();
final _focusNode = FocusNode();   // requestFocus() pour y placer le curseur

@override
void dispose() {
  _controller.dispose();
  _focusNode.dispose();
  super.dispose();
}
```

`dispose` vide la liste des listeners et interdit tout usage ultérieur. Sans lui, un contrôleur rangé dans un objet plus durable que l'écran peut retenir par ses listeners un `State` démonté. Même règle pour tout `ChangeNotifier`, `FocusNode` compris.

Aucune erreur n'est levée, aucun avertissement n'est émis, rien ne change à l'écran.

Un `TextFormField` sans contrôleur fonctionne aussi : `validator` et `onSaved` suffisent souvent.

Le clavier réduit la hauteur disponible

Le `Scaffold` retire la hauteur du clavier de la contrainte de son `body` (`resizeToAvoidBottomInset`, `true` par défaut). Un contenu de hauteur fixe se retrouve masqué.

Approche	Effet
`ListView` pour les champs	le contenu défile, le champ actif reste atteignable
`SingleChildScrollView`	équivalent pour un contenu non listé
`resizeToAvoidBottomInset: false`	le `body` garde sa hauteur, le clavier le recouvre

Le choix du conteneur règle le problème. Ce n'est pas une option à activer.

Lire couleurs et styles dans `Theme.of(context)`

Compose	Flutter
`MaterialTheme.colorScheme.secondaryContainer`	`Theme.of(context).colorScheme.secondaryContainer`
`MaterialTheme.typography.labelMedium`	`Theme.of(context).textTheme.labelMedium`
`Text(maxLines = 2, overflow = TextOverflow.Ellipsis)`	`Text(maxLines: 2, overflow: TextOverflow.ellipsis)`

Les color roles et les styles portent les mêmes noms qu'en Material 3 Compose. Un style de `textTheme` est un `TextStyle?` : `labelMedium?.copyWith(color: …)` en change une propriété.

Une couleur en dur ignore le thème et le mode sombre, sujet de la séance 3.

Côté Compose

Notion	Flutter	Compose
état local	champ du `State` et `setState`	`remember { mutableStateOf(…) }`
ce qu'une écriture reconstruit	le `build()` entier du `State`	les fonctions qui lisent l'état
enfant sauté	même instance (`const`)	paramètres stables et égaux
largeur d'un enfant de `Row`	unbounded : un `Text` long déborde	l'espace restant : il passe à la ligne
répartir l'espace	`Expanded(flex: 2)`	`Modifier.weight(2f)`
texte saisi	`TextEditingController`, à libérer	`remember { mutableStateOf("") }`
focus	`FocusNode`, à libérer	`FocusRequester`

Rendre un TP sur son fork

Le dépôt du cours est en lecture seule. Chacun le forke une fois, depuis `github.com/mduruisseau/pantry-2026`, bouton **Fork**.

```
git clone https://github.com/mduruisseau/pantry-2026.git
cd pantry-2026
git remote add perso https://github.com/<compte>/pantry-2026.git

git switch session-02-start
git switch -c tp02-<nom>
…
git add -A && git commit -m "TP 2"
git push perso tp02-<nom>
```

Les séances suivantes arrivent par `git fetch origin`. GitHub refuse le mot de passe du compte pour un push en HTTPS : un jeton d'accès personnel ou une clé SSH.

Quatre TP tirés au sort comptent pour 20 %

Règle	Détail
ce qui est vérifié	les critères de l'exercice prioritaire, section « Rendu » du `TP.md`
échéance	le dernier commit poussé avant la fin de la séance
binôme	chacun pousse sur son fork ; un rendu commun sur demande, `tp02-<nom1>-<nom2>`
tirage	quatre séances parmi les séances 2 à 10, après la séance 10
absence	une séance tirée et manquée vaut zéro, sauf absence justifiée

Pousser la branche de départ ou le corrigé ne valide rien.

TP 2 : fiche produit et formulaire

60 minutes en binôme

```
git switch session-02-start
git switch -c tp02-<nom>
flutter pub get
flutter run -d chrome
```

Énoncé complet dans `TP.md`, maquette dans `design/fiche-produit.svg`.

La fiche produit d'abord. Le formulaire peut être terminé hors séance.

Points de relecture

- les **six** produits mockés : le rendu tient-il au-delà du premier ?
- un champ absent : ligne supprimée, texte de repli, ou valeur par défaut du modèle ?
- le sélecteur de date : contrôle manuel, ou `FormField<DateTime>` ?
- les contrôleurs : qui les libère dans `dispose` ?

Erreurs fréquentes

Erreur	Effet
`ListView` dans une `Column`, sans `Expanded`	liste non peinte, « Vertical viewport was given unbounded height »
`Text` long dans une `Row`, sans `Expanded`	rayures, et `maxLines` ne tronque rien
`setState` appelé pendant un `build`	« setState() or markNeedsBuild() called during build. »
`Colors.grey` au lieu d'un color role	une couleur qui ne suit ni le thème ni le mode sombre
`main()` ou un champ du `State` modifié, puis hot reload	rien ne change : hot restart (`R`)

Séance 3

Listes, navigation, thème et transitions.

Une question à garder de côté : dans une application à plusieurs écrans, comment un compteur placé dans l'AppBar connaîtrait-il le produit affiché ailleurs ?

`setState` n'y répond pas.

Questions

Pointeurs du cours 1

Ressource	Adresse
Contraintes	`docs.flutter.dev/ui/layout/constraints` · 29 exemples
Flutter pour développeurs Compose	`docs.flutter.dev/flutter-for/compose-devs`
Catalogue de widgets	`docs.flutter.dev/ui/widgets`
Composants Material 3	`docs.flutter.dev/ui/widgets/material`
Widget Inspector	`docs.flutter.dev/tools/devtools/inspector`

Pointeurs du cours 2

Ressource	Adresse
`Form` et validation	`api.flutter.dev/flutter/widgets/Form-class.html`
Cookbook · formulaires	`docs.flutter.dev/cookbook/forms`
`showDatePicker`	`api.flutter.dev/flutter/material/showDatePicker.html`
`Theme`	`api.flutter.dev/flutter/material/Theme-class.html`
Dépôt du cours	`github.com/mduruisseau/pantry-2026` → `session-02-start`